

Device-Simulation GUI — Architecture Spec v0.2.9

Delta

Plateau-Measurement-Integrity Contract, NEEDS_RECHECK State, and the
Audit-Correction / Supersession Pattern

2026-08-23

Scope. This delta specifies three items flagged for v0.2.9 that are not yet in the v0.2.8/v0.2.9 master: (1) the *Plateau-Measurement-Integrity Contract* (PMIC); (2) the explicit NEEDS_RECHECK verification state and its transitions; (3) the audit-correction / supersession pattern for defective-but-preserved evidence. It is written to be folded into the master spec’s verification-checkpoint chapter. It is grounded in a concrete defect resolved in the backend (Liang-1997 SHE-BTE reproduction, 71×58 node, 2026-08-23); that case is used as the worked example so the contract is testable, not aspirational.

1 Motivating defect (normative example)

A convergence observable (rate G) was accepted as “plateaued” when the relative spread of its last four values fell below a gate. The solver stopped each continuation *chunk* on its own residual criterion `tol_phi`. When the solution was already near its fixed point, a chunk stopped after a *single* iteration; the “last four” then spanned one value, whose spread is identically 0 — a **degenerate false pass**. The physical value was fine; the *measurement of acceptance* was not. The GUI must make this class of error unrepresentable.

2 Plateau-Measurement-Integrity Contract (PMIC)

Intent. A plateau verdict may be computed only over genuine, independent samples of the observable, accumulated across solver invocations, and never manufactured by an early solver stop.

2.1 Definitions

Real iteration

one outer fixed-point update that actually advanced the observable; identified by a strictly increasing absolute iteration index `abs_iter` on the node’s cumulative trajectory.

Cumulative window

the last $W=4$ *real* iterations of the node’s trajectory, taken across chunk boundaries — *not* the last four of any single chunk.

Rolling buffer

the persisted, ordered list of the most recent $\leq W$ real samples $\{(\text{abs_iter}, \text{observables})\}$, carried between solver invocations in the node sidecar.

`plateau_evaluable`

a boolean: **false** until the buffer holds W genuine samples.

2.2 Normative rules (MUST)

1. The plateau gate is evaluated over the cumulative window, not a chunk-local window.
2. `plateau_evaluable` is **false** while $|\text{buffer}| < W$; a node MUST NOT be certified while not evaluable, *even if* a chunk-local spread would be 0.
3. The solver-stopping residual (`tol_phi`) is **decoupled** from acceptance: it may end a chunk, but MUST NOT short-circuit or stand in for the observable plateau decision.
4. The production spread statistic and thresholds are **fixed** and identical to the backend: $\text{spread}(x) = (\max x - \min x) / \bar{x}$, gate $\leq 3 \times 10^{-4}$ on *both* rate observables, residual floored $\leq 1.5 \times 10^{-3}$. The GUI MUST source these from one shared definition, never redefine them.
5. **Buffer integrity (fail-closed)**. On resume, `abs_iter` across the buffer MUST be strictly increasing (hence duplicate-free). A violation aborts certification (a duplicated index could fabricate an artificial plateau); it is a hard error, not a warning.
6. **Provenance**. Each certification records `buffer_origin` $\in \{\text{persisted_sidecar}, \text{npz_terminal_reconstruction}, \text{empty_fresh}\}$, so the evidence path that certified the gate is auditable after the fact.
7. A spread of 0 is legitimate *iff* `plateau_evaluable` and the window holds W distinct real iterations (e.g. four independent re-solves re-confirming one fixed point); it is forbidden otherwise.

2.3 Contract surface (data the GUI persists per node)

```
sidecar.plateau = {
  buffer: [ {abs_iter, <observables...>, residual}, ... up to W ],
  plateau_evaluable: bool, # false until |buffer| == W
  passed: bool, # gate over the cumulative window
  buffer_origin: enum, # persisted_sidecar | npz_terminal_reconstruction |
    empty_fresh
  window_iters: [abs_iter...], # the exact indices used (independently checkable)
}
```

2.4 Checkpoint hook

The plateau checkpoint (the master’s convergence-ladder rung for a node) reports PASS only when `plateau_evaluable` $\wedge$ `passed`; it reports NOT_EVALUABLE (a distinct, non-failing, non-passing status) while the window is short; and it reports FAIL when evaluable and the gate is not met. NOT_EVALUABLE MUST drive “continue accumulating”, never “accept”.

3 The needs_recheck verification state

Intent. A node whose *acceptance evidence* is invalid — as opposed to a node that has genuinely failed or genuinely passed — occupies a first-class state that blocks downstream use without asserting a scientific failure.

3.1 State set and transitions

from	event	to
ACCUMULATING	<code>plateau_evaluable</code> $\wedge$ <code>passed</code>	CERTIFIED
ACCUMULATING	evaluable $\wedge$ $\neg$ passed, cap reached	FAILED
CERTIFIED	audit finds evidence degenerate/ambiguous	NEEDS_RECHECK
NEEDS_RECHECK	recheck yields evaluable genuine window, passed	CERTIFIED
NEEDS_RECHECK	recheck evaluable $\wedge$ $\neg$ passed	FAILED
NEEDS_RECHECK	cannot establish a valid window under policy	AUDIT_INCOMPLETE

3.2 Normative rules (MUST)

1. NEEDS_RECHECK is neither PASS nor FAIL; the GUI MUST render it as its own badge and MUST block any campaign-level verdict (e.g. a GREEN/YELLOW classification, or an extrapolation) that depends on the node until it leaves this state.
2. Entering NEEDS_RECHECK MUST NOT overwrite the original recorded metrics row; the original (defective) record is preserved verbatim (see §4).
3. A recheck MUST reuse the identical run definition (mesh, grids, physics, solver settings, thresholds) and change *only* the acceptance measurement; it carries a new `run_id` and names the prior checkpoint as `parent`. A pure safety cap (max continuation chunks) may be raised and MUST be labeled as such, never as a scientific-gate change.
4. Reconstruction subtlety: a short terminal chunk does *not* automatically imply NEEDS_RECHECK. If $\geq W$ genuine cumulative samples are retrievable (persisted buffer, or a full-window terminal reconstruction) the node is evaluable; NEEDS_RECHECK is entered only when a valid window is unretrievable or ambiguous.

4 Audit-correction / supersession pattern

Intent. Corrections are additive and immutable: nothing that was recorded is edited or deleted; a superseding record points back to what it replaces, and the chain is independently verifiable by hash.

4.1 Normative rules (MUST)

1. **Preserve.** The original artifact (data checkpoint, metrics row, generated report) is kept byte-for-byte under an archival name; its content hash is recorded.
2. **Supersede, don't overwrite.** A new *audit-correction record* is emitted as a separate artifact carrying: `original_status`, `audit_status`, `reason`, the superseded artifact's `sha256`, the new `run_id/parent`, and the corrected decision. The record itself gets a `sha256`.
3. **Derive, don't mutate.** A reconstructed or corrected decision is emitted as its own artifact; source data files are never rewritten to “fix” a metric.
4. **Label re-derived verdicts.** A classification recomputed after a measurement repair MUST be visibly labeled as re-derived (e.g. “YELLOW — re-derived after plateau-bookkeeping repair”) so it cannot be confused with the earlier defective output. Reports MUST present *scientific reproduction* and *numerical parity* as separate statements.

5. **Lineage.** If a corrected node was a warm-start parent of another accepted node, the record states whether the corrected value moved materially; if it did not, dependent nodes are *not* regenerated, and the record says so explicitly.

4.2 Audit-correction record (schema)

```
audit_correction = {
  node, original_status, audit_status, # e.g. PASS -> NEEDS_RECHECK
  reason, # e.g. "degenerate_plateau_window"
  superseded_artifact, superseded_sha256,
  run_id, parent_checkpoint_sha256,
  resolution: { new_status, evidence, window_iters, buffer_origin },
  sha256 # of this record
}
```

5 Worked example (normative acceptance test for the GUI)

The GUI’s verification layer **MUST** reproduce this sequence for the 71×58 node:

1. Original acceptance: terminal chunk ran 1 real iteration; chunk-local spread = 0 $\Rightarrow$ *defective* CERTIFIED. Audit reclassifies to NEEDS_RECHECK; original row preserved; audit-correction record emitted (**reason=degenerate_plateau_window**).
2. Recheck (new **run_id**, same run definition, safety cap raised only): the node early-stops 1 real iter/chunk. PMIC holds NOT_EVALUABLE at cumulative $n=1, 2, 3$ (indices 359, 360, 361) and certifies at $n=4$ (index 362): four distinct re-solves, both spreads = 0 (G_{ii} spread = 0; G_{tot} spread = 0), residual floored. NEEDS_RECHECK $\rightarrow$ CERTIFIED.
3. Campaign verdict recomputed on the certified nodes; classification labeled “re-derived after plateau-bookkeeping repair”; scientific reproduction (significant non-monotone reversal) and numerical parity ($\pm 2\%$ archival) reported separately; lineage note records the parent value was unchanged, so the dependent node is not regenerated.

Any GUI build that would let step 1 stand as a pass, that mutates the original row in step 1, or that omits the re-derived label / parity separation in step 3, **fails** this acceptance test.